SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Scenario-Based Task (Medium)
Assessment Tool Weightage: 20%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF	. BL4 – Analyze
2	A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach	. BL6 – Create
3	Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.	BL5 – Evaluate
4	A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.	BL6 – Create
5	A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.	BL4 – Analyze
6	Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.	BL3 – Apply
7	A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.	BL4 – Analyze

8	A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.	BL5 – Evaluate
9	For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.	BL6 – Create
10	A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.	BL5 – Evaluate

---

Code of Conduct:

*I _ **M O N I K A . R (1 9 2 3 2 4 2 8 1)** _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:
Monika.R

QUESTION: 1

A university stores student and course data in a single table. Analyze the structure and identify normalization issues, then redesign it up to 3NF

BEFORE NORMALIZATION:

StudentID	StudentName	Dept	CourseID	CourseName	Faculty	FacultyPhone
S101	Rahul	CSE	C101	DBMS	Dr. Kumar	9876543210
S101	Rahul	CSE	C102	Operating System	Dr. Priya	9123456780
S102	Anitha	IT	C101	DBMS	Dr. Kumar	9876543210

Problems in the Table

1. Data redundancy (Student details repeated)
2. Course details repeated.
3. Faculty details repeated.
4. Update anomaly.
5. Insertion anomaly.
6. Deletion anomaly.

Functional Dependencies

- StudentID → StudentName, Dept
- CourseID → CourseName, Faculty
- Faculty → FacultyPhone

Candidate Key:

(StudentID, CourseID)

Step 1: First Normal Form (1NF)

- Remove repeating groups.
- Store atomic values only.

StudentID	StudentName	Dept	CourseID	CourseName	Faculty	FacultyPhone
S101	Rahul	CSE	C101	DBMS	Dr. Kumar	9876543210
S101	Rahul	CSE	C102	Operating System	Dr. Priya	9123456780
S102	Anitha	IT	C101	DBMS	Dr. Kumar	9876543210

(Table already satisfies 1NF.)

Step 2: Second Normal Form (2NF)

Remove partial dependency.

StudentID	StudentName	Dept
S101	Rahul	CSE
S102	Anitha	IT

COURSE

CourseID	**CourseName**	**Faculty**
C101	DBMS	Dr. Kumar
C102	Operating System	Dr. Priya

ENROLLMENT

StudentID	**CourseID**
S101	C101
S101	C102
S102	C101

Step 3: Third Normal Form (3NF)

Remove transitive dependency.

FACULTY

FacultyID	FacultyName	FacultyPhone
F101	Dr. Kumar	9876543210
F102	Dr. Priya	9123456780

COURSE

CourseID	CourseName	FacultyID
C101	DBMS	F101
C102	Operating System	F102

Final 3NF Schema

Student(StudentID, StudentName, Dept)

Faculty(FacultyID, FacultyName, FacultyPhone)

Course(CourseID, CourseName, FacultyID)

Enrollment(StudentID, CourseID)

Foreign Keys:

StudentID → Student

CourseID → Course

Advantages

- No redundancy
- No update anomaly
- No insertion anomaly
- No deletion anomaly
- Better consistency
- Improved data integrity

QUESTION: 2

A company database contains employee and department details. Design appropriate SQL queries to retrieve employees working in multiple departments and justify your approach

ANSWER:

TABLES:

Employee

EmpID	EmpName
101	John
102	David
103	Anu
104	Priya

Department

DeptID	DeptName
D1	HR
D2	Sales
D3	IT

EmployeeDepartment

EmpID	DeptID
101	D1
101	D2
102	D2
103	D1
103	D2
103	D3
104	D3

Query Explanation

- JOIN combines Employee and EmployeeDepartment tables.
- GROUP BY groups records employee-wise.
- COUNT() counts number of departments.
- HAVING filters employees working in more than one department.

Justification

- JOIN avoids data duplication.
- GROUP BY performs aggregation.
- HAVING filters grouped data.
- Efficient and scalable.
- Suitable for many-to-many relationships.

SQL QUERIES:

```
mysql> CREATE DATABASE CompanyDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE CompanyDB;
Database changed
mysql> CREATE TABLE Employee(EmpID INT PRIMARY KEY, EmpName VARCHAR(50));
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Department(DeptID VARCHAR(5) PRIMARY KEY, DeptName VARCHAR(30));
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE EmployeeDepartment(EmpID INT, DeptID VARCHAR(5), PRIMARY KEY(EmpID, DeptID), FOREIGN KEY(EmpID) REFERENCES Employee(EmpID), FOREIGN KEY(DeptID) REFERENCES Department(De
ptID));
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Employee VALUES(101,'John');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Employee VALUES(102,'David');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Employee VALUES(103,'Anu');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO Employee VALUES(104,'Priya');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Department VALUES('D1','HR');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Department VALUES('D2','Sales');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Department VALUES('D3','IT');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Employee;
+-------+---------+
| EmpID | EmpName |
+-------+---------+
|   101 | John    |
|   102 | David   |
|   103 | Anu     |
|   104 | Priya   |
+-------+---------+
4 rows in set (0.00 sec)
mysql> SELECT * FROM Department;
+---------+-----------+
| DeptID | DeptName |
+---------+-----------+
| D1      | HR        |
| D2      | Sales     |
| D3      | IT        |
+---------+-----------+
3 rows in set (0.00 sec)

mysql> INSERT INTO EmployeeDepartment VALUES(101,'D1');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO EmployeeDepartment VALUES(101,'D2');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO EmployeeDepartment VALUES(102,'D2');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO EmployeeDepartment VALUES(103,'D1');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO EmployeeDepartment VALUES(103,'D2');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO EmployeeDepartment VALUES(103,'D3');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO EmployeeDepartment VALUES(104,'D3');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM EmployeeDepartment;
+-------+--------+
| EmpID | DeptID |
+-------+--------+
|   101 | D1     |
|   103 | D1     |
|   101 | D2     |
|   102 | D2     |
|   103 | D2     |
|   103 | D3     |
|   104 | D3     |
+-------+--------+
7 rows in set (0.00 sec)
```

```
mysql> SELECT e.EmpID, e.EmpName, COUNT(ed.DeptID) AS Department_Count FROM Employee e JOIN EmployeeDepartment ed ON e.EmpID=ed.EmpID GROUP BY e.EmpID,e.Emp
Name HAVING COUNT(ed.DeptID)>1;
+-------+---------+------------------+
| EmpID | EmpName | Department_Count |
+-------+---------+------------------+
|   101 | John    |                2 |
|   103 | Anu     |                3 |
+-------+---------+------------------+
2 rows in set (0.01 sec)

mysql> SELECT e.EmpID, e.EmpName, d.DeptName FROM Employee e JOIN EmployeeDepartment ed ON e.EmpID=ed.EmpID JOIN Department d ON ed.DeptID=d.DeptID ORDER BY
 e.EmpID;
+-------+---------+----------+
| EmpID | EmpName | DeptName |
+-------+---------+----------+
|   101 | John    | HR       |
|   101 | John    | Sales    |
|   102 | David   | Sales    |
|   103 | Anu     | HR       |
|   103 | Anu     | Sales    |
|   103 | Anu     | IT       |
|   104 | Priya   | IT       |
+-------+---------+----------+
7 rows in set (0.01 sec)
```

QUESTION:3

Given a relation with multiple functional dependencies, determine the candidate keys and check whether the relation satisfies BCNF. If not, decompose it.

ANSWER:

RELATION: R(A,B,C,D)

Functional Dependencies

A → B

B → C

C → D

Step 1: Find Closure

A+

A+ = A

A → B = A,B

B → C = A,B,C

C → D = A,B,C,D

Therefore, A+ = {A,B,C,D}

Hence, Candidate Key = A

Prime Attribute A

Non-prime Attributes B C D

BCNF Check

BCNF Rule:

For every functional dependency, **X → Y**

X must be a super key.

Dependency 1 A → B

A is Candidate Key

✓ Satisfies BCNF

Dependency 2 B → C

B is NOT Candidate Key

✗ Violates BCNF

Dependency 3 C → D

C is NOT Candidate Key

✗ Violates BCNF

BCNF Decomposition

Relation 1 R1(B,C)

FD B → C

Key B

BCNF ✓

Relation 2 R2(C,D)

FD C → D

Key C

BCNF ✓

Relation 3 R3(A,B)

FD A → B

Key A

BCNF ✓

Final BCNF Relations

R1(B,C)

R2(C,D)

R3(A,B)

Advantages of BCNF

- Eliminates redundancy.
- Removes update anomalies.
- Prevents insertion and deletion anomalies.
- Ensures every determinant is a candidate key.
- Improves data consistency and integrity.

QUESTION: 4

A banking system requires secure data access. Design views and constraints to ensure data security and integrity for different types of users.

ANSWER:

```
mysql> CREATE DATABASE BankingDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE BankingDB;
Database changed
mysql> CREATE TABLE Customer(CustomerID INT PRIMARY KEY, CustomerName VARCHAR(50), AccountNo VARCHAR(20) UNIQUE, Balance DECIMAL(10,2) CHECK(Balance>=0), Br
anch VARCHAR(30));
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Customer VALUES(101,'Rahul','ACC101',50000,'Chennai');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Customer VALUES(102,'Anitha','ACC102',75000,'Coimbatore');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Customer VALUES(103,'Karthik','ACC103',65000,'Madurai');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT * FROM Customer;
+------------+--------------+-----------+----------+------------+
| CustomerID | CustomerName | AccountNo | Balance  | Branch     |
+------------+--------------+-----------+----------+------------+
|        101 | Rahul        | ACC101    | 50000.00 | Chennai    |
|        102 | Anitha       | ACC102    | 75000.00 | Coimbatore |
|        103 | Karthik      | ACC103    | 65000.00 | Madurai    |
+------------+--------------+-----------+----------+------------+
3 rows in set (0.00 sec)

mysql> CREATE VIEW Customer_View AS SELECT CustomerID,CustomerName,AccountNo,Balance FROM Customer;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT * FROM Customer_View;
+------------+--------------+-----------+----------+
| CustomerID | CustomerName | AccountNo | Balance  |
+------------+--------------+-----------+----------+
|        101 | Rahul        | ACC101    | 50000.00 |
|        102 | Anitha       | ACC102    | 75000.00 |
|        103 | Karthik      | ACC103    | 65000.00 |
+------------+--------------+-----------+----------+
3 rows in set (0.00 sec)
```

Integrity Constraints Used

- PRIMARY KEY(CustomerID)
- UNIQUE(AccountNo)
- CHECK(Balance>=0)
- NOT NULL (for mandatory fields)
- VIEW hides confidential information from unauthorized users.

Advantages

- Provides secure access.
- Prevents duplicate account numbers.
- Prevents negative balance.
- Improves data integrity.
- Restricts unauthorized access.

QUESTION: 5

A sales database contains redundant data leading to anomalies. Analyze the schema and apply normalization up to 3NF or BCNF to eliminate redundancy.

ANSWER:

Unnormalized Relation

OrderID	CustomerID	CustomerName	ProductID	ProductName	SupplierName
O101	C101	Rahul	P101	Laptop	Dell
O102	C101	Rahul	P102	Mouse	Logitech
O103	C102	Anitha	P101	Laptop	Dell

Functional Dependencies

- CustomerID → CustomerName
- ProductID → ProductName, SupplierName
- OrderID → CustomerID, ProductID

Candidate Key OrderID

1NF

OrderID	CustomerID	CustomerName	ProductID	ProductName	SupplierName
O101	C101	Rahul	P101	Laptop	Dell
O102	C101	Rahul	P102	Mouse	Logitech
O103	C102	Anitha	P101	Laptop	Dell

2NF

CUSTOMER

CustomerID	CustomerName
C101	Rahul
C102	Anitha

PRODUCT

ProductID	ProductName	SupplierName
P101	Laptop	Dell
P102	Mouse	Logitech

ORDER

OrderID	CustomerID	ProductID
O101	C101	P101
O102	C101	P102
O103	C102	P101

3NF

CUSTOMER

CustomerID	CustomerName
C101	Rahul
C102	Anitha

SUPPLIER

SupplierID	SupplierName
S101	Dell
S102	Logitech

PRODUCT

ProductID	ProductName	SupplierID
P101	Laptop	S101
P102	Mouse	S102

ORDER

OrderID	CustomerID	ProductID
O101	C101	P101
O102	C101	P102
O103	C102	P101

Final 3NF Schema

Customer(CustomerID, CustomerName)

Supplier(SupplierID, SupplierName)

Product(ProductID, ProductName, SupplierID)

Orders(OrderID, CustomerID, ProductID)

Advantages

- Eliminates redundancy.

- Removes update, insertion, and deletion anomalies.
- Improves consistency.
- Ensures data integrity.
- Simplifies maintenance.

QUESTION: 6

Given a dataset of customers and orders, write SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

ANSWER:

SQL queries using joins and subqueries to retrieve meaningful information such as frequent customers.

```
mysql> CREATE DATABASE CustomerDB;
Query OK, 1 row affected (0.02 sec)

mysql> USE CustomerDB;
Database changed
mysql> CREATE TABLE Customer(CustomerID INT PRIMARY KEY, CustomerName VARCHAR(50), City VARCHAR(30));
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE Orders(OrderID INT PRIMARY KEY, CustomerID INT, Amount DECIMAL(10,2), FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID));
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Customer VALUES(101,'Rahul','Chennai');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Customer VALUES(102,'Anitha','Coimbatore');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Customer VALUES(103,'Karthik','Madurai');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Customer;
+------------+--------------+------------+
| CustomerID | CustomerName | City       |
+------------+--------------+------------+
|        101 | Rahul        | Chennai    |
|        102 | Anitha       | Coimbatore |
|        103 | Karthik      | Madurai    |
+------------+--------------+------------+
3 rows in set (0.00 sec)

mysql> INSERT INTO Orders VALUES(1,101,5000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Orders VALUES(2,101,8000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Orders VALUES(3,102,6000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Orders VALUES(4,101,7000);
```

```
mysql> INSERT INTO Orders VALUES(5,103,4000);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Orders;
+---------+------------+---------+
| OrderID | CustomerID | Amount  |
+---------+------------+---------+
|       1 |        101 | 5000.00 |
|       2 |        101 | 8000.00 |
|       3 |        102 | 6000.00 |
|       4 |        101 | 7000.00 |
|       5 |        103 | 4000.00 |
+---------+------------+---------+
5 rows in set (0.00 sec)

mysql> SELECT c.CustomerID,c.CustomerName,o.OrderID,o.Amount FROM Customer c JOIN Orders o ON c.CustomerID=o.CustomerID;
+------------+--------------+---------+---------+
| CustomerID | CustomerName | OrderID | Amount  |
+------------+--------------+---------+---------+
|        101 | Rahul        |       1 | 5000.00 |
|        101 | Rahul        |       2 | 8000.00 |
|        101 | Rahul        |       4 | 7000.00 |
|        102 | Anitha       |       3 | 6000.00 |
|        103 | Karthik      |       5 | 4000.00 |
+------------+--------------+---------+---------+
5 rows in set (0.00 sec)

mysql> SELECT CustomerName FROM Customer WHERE CustomerID IN (SELECT CustomerID FROM Orders GROUP BY CustomerID HAVING COUNT(OrderID)>2);
+--------------+
| CustomerName |
+--------------+
| Rahul        |
+--------------+
1 row in set (0.01 sec)
```

Advantages

- JOIN combines customer and order details.
- Subquery identifies frequent customers.
- GROUP BY groups customer orders.
- HAVING filters customers with more than two orders.

QUESTION: 7

A relation contains multi-valued dependencies. Analyze the structure and decompose it into 4NF, explaining the reasoning.

ANSWER:

Relation: StudentID, Hobby, Language

StudentID	Hobby	Language
101	Cricket	English
101	Cricket	Hindi
101	Music	English
101	Music	Hindi
102	Reading	Tamil
102	Reading	English

Multi-Valued Dependencies

StudentID →→ Hobby

StudentID →→ Language

Decomposition into 4NF

STUDENT_HOBBY

StudentID	Hobby
101	Cricket
101	Music
102	Reading

STUDENT_LANGUAGE

StudentID	**Language**
101	English
101	Hindi
102	Tamil
102	English

Final 4NF Schema

Student_Hobby(StudentID, Hobby)

Student_Language(StudentID, Language)

Reasoning

- The multi-valued dependencies are separated into independent relations.
- Each table contains only one independent multi-valued fact.
- Redundancy is eliminated.
- Update, insertion, and deletion anomalies are removed.

- The schema satisfies Fourth Normal Form (4NF).

Advantages

- Eliminates redundancy caused by multi-valued dependencies.
- Improves consistency.
- Simplifies maintenance.
- Preserves data integrity.
- Achieves 4NF.

QUESTION: 8

A database designer used improper constraints, leading to inconsistent data. Evaluate the schema and suggest appropriate integrity constraints.

ANSWER:

```
mysql> CREATE DATABASE CollegeDBSS;
Query OK, 1 row affected (0.01 sec)

mysql> USE CollegeDBSS;
Database changed
mysql> CREATE TABLE Student(StudentID INT PRIMARY KEY, StudentName VARCHAR(50) NOT NULL, Email VARCHAR(100) UNIQUE, Age INT CHECK(Age>=18), Department VARCH
AR(30) NOT NULL);
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Student VALUES(101,'Rahul','rahul@gmail.com',20,'CSE');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Student VALUES(102,'Anitha','anitha@gmail.com',21,'IT');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Student VALUES(103,'Karthik','karthik@gmail.com',22,'ECE');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Student;
+-----------+-------------+-----------------+-----+------------+
| StudentID | StudentName | Email           | Age | Department |
+-----------+-------------+-----------------+-----+------------+
|       101 | Rahul       | rahul@gmail.com |  20 | CSE        |
|       102 | Anitha      | anitha@gmail.com |  21 | IT         |
|       103 | Karthik     | karthik@gmail.com |  22 | ECE        |
+-----------+-------------+-----------------+-----+------------+
3 rows in set (0.00 sec)
```

Integrity Constraints Used

- PRIMARY KEY(StudentID)
- NOT NULL(StudentName)
- UNIQUE(Email)
- CHECK(Age >= 18)
- NOT NULL(Department)

Advantages

- Prevents duplicate Student IDs.
- Prevents duplicate email addresses.
- Ensures mandatory fields are not empty.
- Restricts invalid age values.
- Maintains data consistency and integrity.

QUESTION:9

For a given relational schema, analyze the presence of join dependencies and decompose it into 5NF to remove redundancy.

ANSWER:

RELATION: SupplierID, PartID, ProjectID

Sample Relation

SupplierID	PartID	ProjectID
S1	P1	J1
S1	P2	J1
S2	P1	J2
S2	P2	J2

Join Dependency (SupplierID, PartID, ProjectID)

The relation contains a join dependency, which may introduce redundancy.

Decomposition into 5NF

SUPPLIER_PART

SupplierID	PartID
S1	P1
S1	P2
S2	P1
S2	P2

SUPPLIER_PROJECT

SupplierID	ProjectID
S1	J1
S2	J2

PART_PROJECT

PartID	ProjectID
P1	J1
P2	J1
P1	J2
P2	J2

Final 5NF Schema

Supplier_Part(SupplierID, PartID)

Supplier_Project(SupplierID, ProjectID)

Part_Project(PartID, ProjectID)

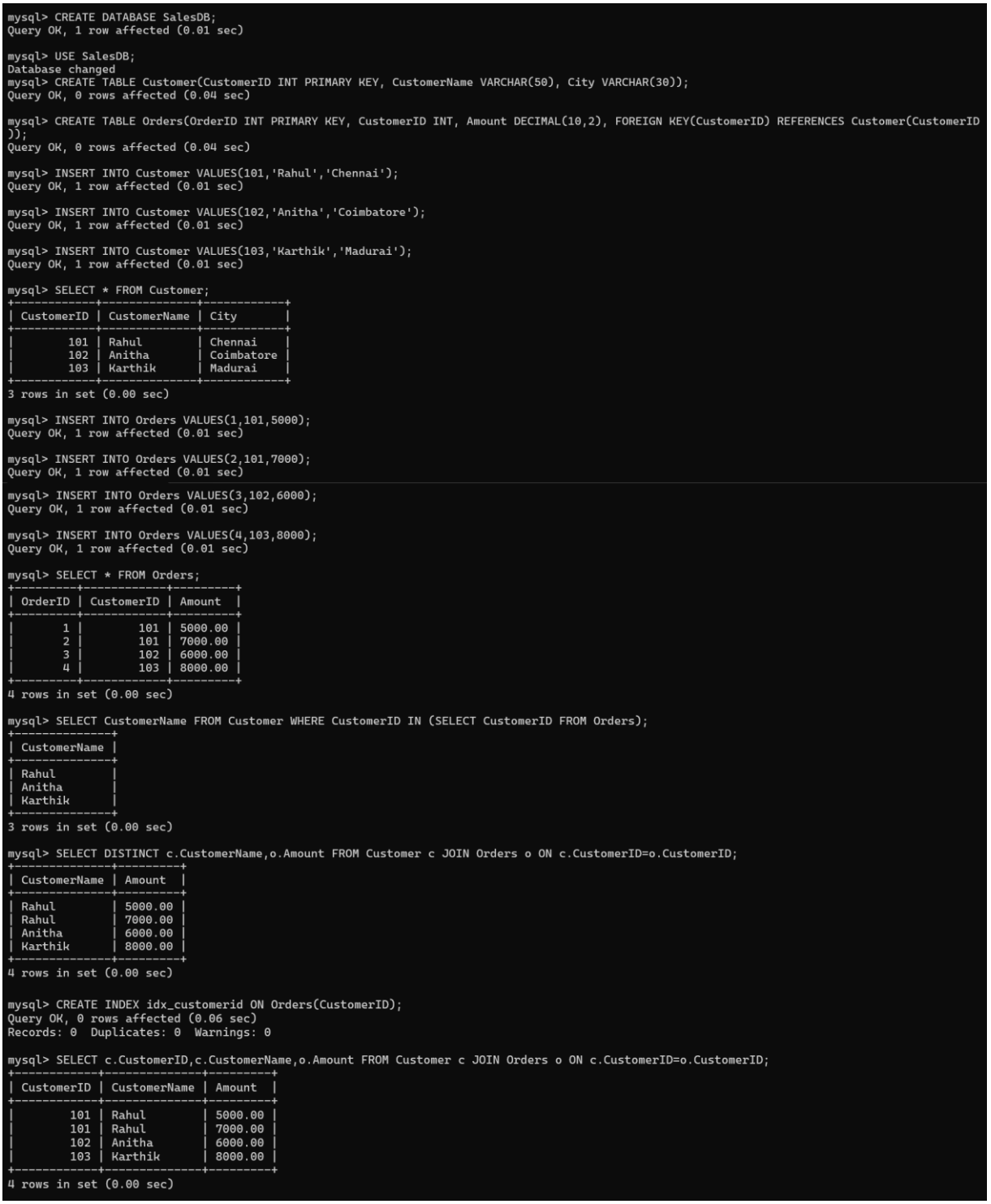
Advantages

- Removes redundancy caused by join dependencies.
- Preserves lossless joins.
- Improves consistency.
- Satisfies Fifth Normal Form (5NF).

QUESTION: 10

A system uses inefficient SQL queries for data retrieval. Analyze the queries and optimize them using joins, indexing concepts, or query restructuring.

ANSWER: SQL queries for data retrieval



```
mysql> CREATE DATABASE SalesDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE SalesDB;
Database changed
mysql> CREATE TABLE Customer(CustomerID INT PRIMARY KEY, CustomerName VARCHAR(50), City VARCHAR(30));
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Orders(OrderID INT PRIMARY KEY, CustomerID INT, Amount DECIMAL(10,2), FOREIGN KEY(CustomerID) REFERENCES Customer(CustomerID
));
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO Customer VALUES(101,'Rahul','Chennai');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Customer VALUES(102,'Anitha','Coimbatore');
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Customer VALUES(103,'Karthik','Madurai');
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Customer;
+------------+--------------+------------+
| CustomerID | CustomerName | City       |
+------------+--------------+------------+
|        101 | Rahul        | Chennai    |
|        102 | Anitha       | Coimbatore |
|        103 | Karthik      | Madurai    |
+------------+--------------+------------+
3 rows in set (0.00 sec)

mysql> INSERT INTO Orders VALUES(1,101,5000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Orders VALUES(2,101,7000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Orders VALUES(3,102,6000);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO Orders VALUES(4,103,8000);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Orders;
+---------+------------+---------+
| OrderID | CustomerID | Amount  |
+---------+------------+---------+
|       1 |        101 | 5000.00 |
|       2 |        101 | 7000.00 |
|       3 |        102 | 6000.00 |
|       4 |        103 | 8000.00 |
+---------+------------+---------+
4 rows in set (0.00 sec)

mysql> SELECT CustomerName FROM Customer WHERE CustomerID IN (SELECT CustomerID FROM Orders);
+--------------+
| CustomerName |
+--------------+
| Rahul        |
| Anitha       |
| Karthik      |
+--------------+
3 rows in set (0.00 sec)

mysql> SELECT DISTINCT c.CustomerName,o.Amount FROM Customer c JOIN Orders o ON c.CustomerID=o.CustomerID;
+--------------+---------+
| CustomerName | Amount  |
+--------------+---------+
| Rahul        | 5000.00 |
| Rahul        | 7000.00 |
| Anitha       | 6000.00 |
| Karthik      | 8000.00 |
+--------------+---------+
4 rows in set (0.00 sec)

mysql> CREATE INDEX idx_customerid ON Orders(CustomerID);
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SELECT c.CustomerID,c.CustomerName,o.Amount FROM Customer c JOIN Orders o ON c.CustomerID=o.CustomerID;
+------------+--------------+---------+
| CustomerID | CustomerName | Amount  |
+------------+--------------+---------+
|        101 | Rahul        | 5000.00 |
|        101 | Rahul        | 7000.00 |
|        102 | Anitha       | 6000.00 |
|        103 | Karthik      | 8000.00 |
+------------+--------------+---------+
4 rows in set (0.00 sec)
```

Optimization Techniques Used

1. Replaced the subquery with a JOIN.
2. Created an INDEX on CustomerID.
3. Used DISTINCT to avoid duplicate customer names.
4. Used PRIMARY KEY and FOREIGN KEY for efficient joins.
5. Retrieved only the required columns instead of using SELECT *.

Advantages

- Faster query execution.
- Reduced execution time.
- Better index utilization.
- Improved database performance.
- Efficient retrieval of customer and order details.

Conclusion

The inefficient SQL query was optimized by using JOIN, INDEXING, and query restructuring, resulting in faster execution, reduced redundancy, and improved overall database performance.

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand